CSE 242 — Assignment 3

Name: Astra Tsai Student ID: 2000926

Question 1: Hard-margin linear SVM

Support vectors. Points **1, 5, and 6** are the closest training points to the maximum-margin line. The positive supporting line $x_1 + x_2 = 2$ touches point 1, $(1, 1)$. The negative supporting line $x_1 + x_2 = 1$ touches points 5 and 6, $(1, 0)$ and $(0, 1)$. All remaining points lie strictly farther from the separator.

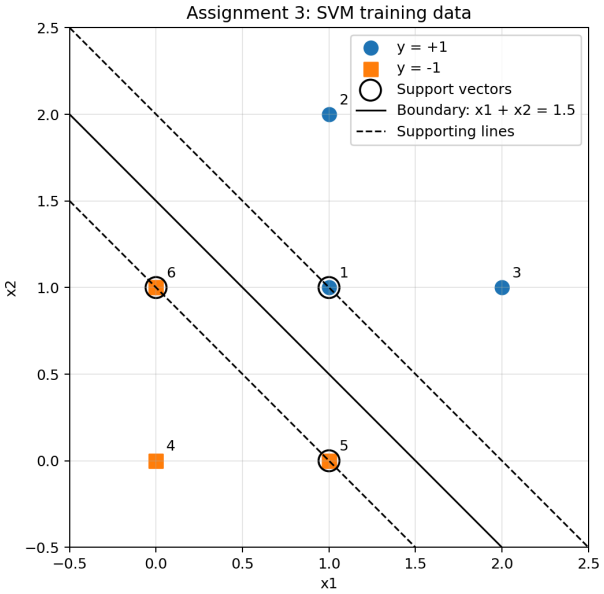


Figure 1: Circled points are the support vectors. The solid line is the decision boundary; dashed lines are the supporting boundaries.

Derivation, including the intercept. In canonical scaling, support vectors satisfy $y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) = 1$. Substituting points 1, 5, and 6 gives

$$\begin{aligned} w_1 + w_2 + b &= 1, \\ w_1 + b &= -1, \\ w_2 + b &= -1. \end{aligned}$$

The last two equations imply $w_1 = w_2$. Subtracting the second equation from the first gives $w_2 = 2$, so $w_1 = 2$ and $b = -3$. Therefore

$$\boldsymbol{w} = \begin{pmatrix} 2 \\ 2 \end{pmatrix}, \quad b = -3, \quad 2x_1 + 2x_2 - 3 = 0, \quad x_2 = -x_1 + \frac{3}{2}.$$

The functional margins $y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b)$, in point-ID order, are $(1, 3, 3, 3, 1, 1)$, so every hard-margin constraint holds.

Geometric margin. The distance from the separator to a closest training point is

$$\gamma = \frac{1}{\|\boldsymbol{w}\|} = \frac{1}{\sqrt{2^2 + 2^2}} = \frac{1}{2\sqrt{2}} \approx 0.353553.$$

The **full width** between supporting lines is $2\gamma = \frac{1}{\sqrt{2}} \approx 0.707107$.

Why the margin is maximal. The negative convex hull contains the midpoint $\boldsymbol{q} = \left(\frac{1}{2}, \frac{1}{2}\right)$ of points 5 and 6; point 1 is $\boldsymbol{p} = (1, 1)$. Their distance is $\|\boldsymbol{p} - \boldsymbol{q}\| = \frac{1}{\sqrt{2}}$. For any separating unit normal, the gap between supporting lines cannot exceed the projection of $\boldsymbol{p} - \boldsymbol{q}$ onto that normal, which is at most this distance. The derived parallel lines attain that bound, proving maximum margin.

Question 2: K-means — implementation and smaller k

Assignment step. For each data row $\boldsymbol{x}_i$, compute the squared Euclidean distance to every centroid and choose

$$a_i = \arg\min_{j \in \{0,\ldots,k-1\}} \left\|\boldsymbol{x}_i - \boldsymbol{\mu}_j\right\|^2.$$

Taking a square root would not change the minimizer. NumPy's `argmin` selects the smallest centroid index in an exact distance tie.

Centroid step. For each nonempty cluster $C_j = \{i : a_i = j\}$, set

$$\boldsymbol{\mu}_j^{\text{new}} = \frac{1}{|C_j|} \sum_{i \in C_j} \boldsymbol{x}_i.$$

If C_j is empty, retain its previous centroid. The update uses a floating-point copy, so it neither mutates the input centroids nor truncates fractional means when the original data have integer type. These policies also handle duplicate initial centroids.

Experiment. Generate 4,000 points using `make_blobs(n_samples=4000, centers=4, cluster_std=0.60, random_state=0)`. A single `np.random.default_rng(0)` supplies the initial data-row samples, with replacement, for $k = 2, 3, 4, 6, 10$ in that order. Run the provided 100-iteration loop, followed by a final assignment step. True blob labels are discarded. All five runs reached a fixed point: every final centroid equals the mean of its assigned points. Colors identify fitted clusters; black crosses mark centroids.

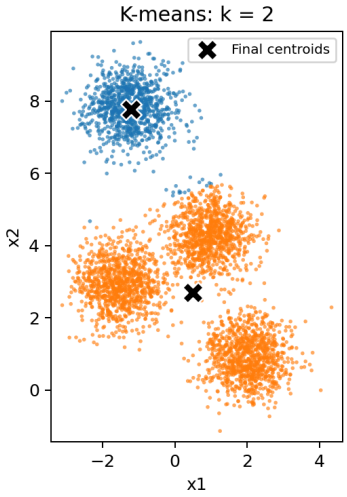


Figure 2: $k = 2$: the upper group is separated; the other three share a centroid.

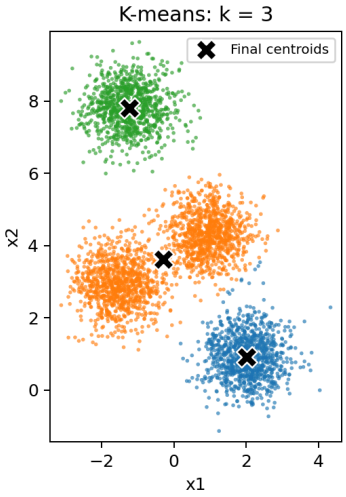


Figure 3: $k = 3$: the two central groups are merged, while the upper and lower groups separate.

These smaller values of k cannot allocate a distinct centroid to every generating group. At $k = 2$, one centroid sits between three visible clouds; at $k = 3$, two nearby clouds share a centroid. The next page shows how extra centroids resolve groups and then subdivide them.

Question 2: K-means — larger k and discussion

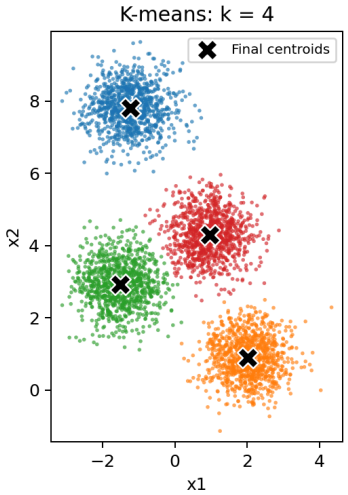


Figure 4: $k = 4$: one centroid per generating group.

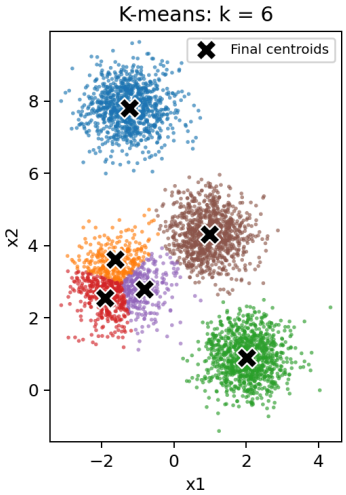


Figure 5: $k = 6$: three centroids divide the left central group.

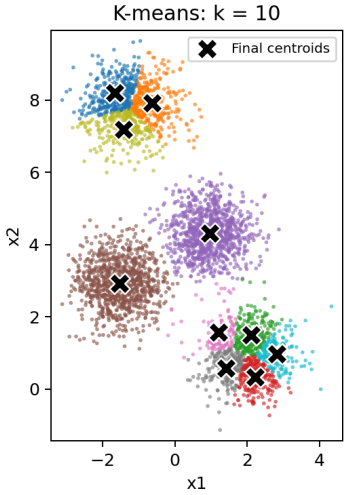


Figure 6: $k = 10$: the upper and lower groups are subdivided further.

Within-cluster sum of squares

k	Sum of squares
2	15,222.212227
3	6,826.126715
4	2,794.241383
6	2,396.788843
10	1,914.786087

Discussion. Increasing k first separates the four generating groups and then partitions individual groups into smaller regions. Here $k = 4$ matches the visible four-cloud structure. At $k = 6$, three centroids fall inside the left central group, leaving one in each other group. At $k = 10$, the upper group receives three centroids and the lower group five, while the two central groups receive one each. This uneven allocation reflects sensitivity to initialization and local optima. The objective decreases across these runs, with a much larger reduction up to $k = 4$ than afterward. Additional centroids can reduce squared error without representing additional generating groups. Separate random initializations do not guarantee monotonic objective values for every possible experiment, and these runs do not establish global optimality.

Question 3: Noisy-label SVM and K-NN repair

Reproducible protocol. Use `np.random.default_rng(0)` to generate training and then independent test data once. Each split has 1,000 observations from $\mathcal{N}((1,1), I)$ with label $+1$, and 1,000 from $\mathcal{N}((-1,-1), 3I)$ with label -1. The matrix $3I$ is the covariance, so each negative feature has standard deviation $\sqrt{3}$.

For each of 20 trials, draw one independent uniform variate per training row and flip it with probability 0.35 for an **original** positive label or 0.20 for an **original** negative label. Selecting all probabilities before flipping prevents double flips. The test features and clean labels remain fixed. Fit a fresh RBF SVM with $C = 1$ and $\gamma = 0.01$ to the noisy training data.

For repair, use the same features and noisy labels as the paired baseline. Compute Euclidean neighbor ordering from training features only, exclude each row itself by setting its diagonal distance to infinity, and select five other points. Exact distance ties favor the smaller original row index; distinct duplicate rows remain eligible. Replace every label simultaneously by the sign of the sum of its five neighbors' original noisy labels. Five binary votes cannot tie. Train another fresh SVM with identical parameters. Neither clean training labels nor test information enters the repair function.

Method / comparison	**Mean accuracy**
SVM with noisy labels	82.795%
SVM after K-NN repair	86.44%
Repair minus baseline	+3.645 pp

Per-trial clean-test accuracies (fractions; the same values are saved in `results/noise_comparison.csv`):

Trial	**Baseline**	**K-NN**	**Trial**	**Baseline**	**K-NN**
1	0.8345	0.8615	11	0.822	0.86
2	0.833	0.872	12	0.819	0.8645
3	0.844	0.8675	13	0.847	0.8645
4	0.8445	0.87	14	0.8415	0.872
5	0.8395	0.8725	15	0.816	0.8715
6	0.841	0.8715	16	0.8285	0.8485
7	0.8405	0.869	17	0.8235	0.8695
8	0.793	0.848	18	0.843	0.87
9	0.824	0.8615	19	0.784	0.8335
10	0.8425	0.8715	20	0.798	0.869

Interpretation. Repair helped in all 20 trials, improving mean accuracy by **3.645 percentage points**. Nearby observations often share a clean class, so majority voting suppresses independent label flips. Both flip probabilities are below 0.5. For example, in a purely positive neighborhood, five independent votes with positive probability 0.65 have a positive majority with probability $\sum_{j=3}^{5} \binom{5}{j} 0.65^j 0.35^{5-j} \approx 0.764831$. Actual neighborhoods can mix classes because the Gaussians overlap. Voting can then erase valid minority labels or spread errors, especially near a boundary or in sparse regions. These results are conditional on one fixed training/test draw and do not guarantee improvement on other datasets.

Outside help. OpenAI Codex (ChatGPT) generated the implementations, mathematical derivation, plots, experiment results, report, and verification checks from the supplied assignment. No external solution sources were used.